SAMSUNG

Samsung Innovation Campus

Generative AI Module

Together for Tomorrow!
Enabling People
Education for Future Generations

Notas de Clase

Generative AI Module

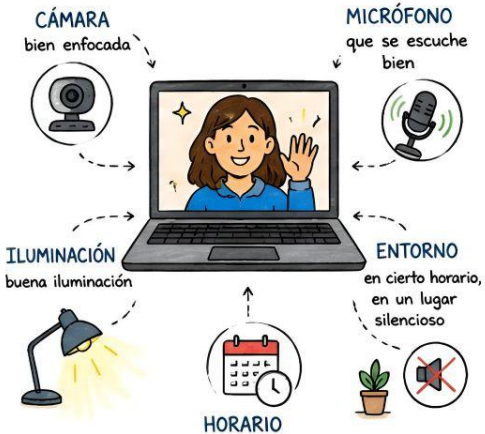
OBJETIVOS DEL CURSO

1 Se lleven como razonar un problema y saber cuando usar AI y cuando no.



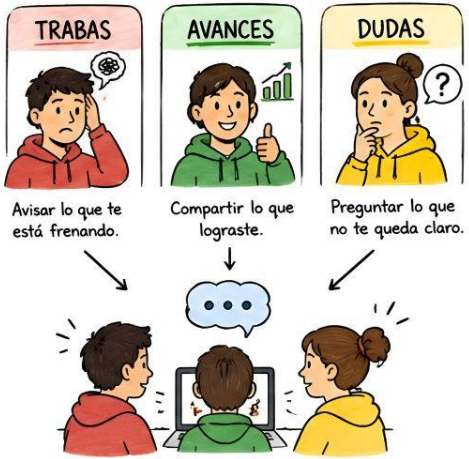
Primero pensar, después decidir.

2 Organizar su vida para tener meets, eso implica tener tu carta de presentacion en buenas condiciones:



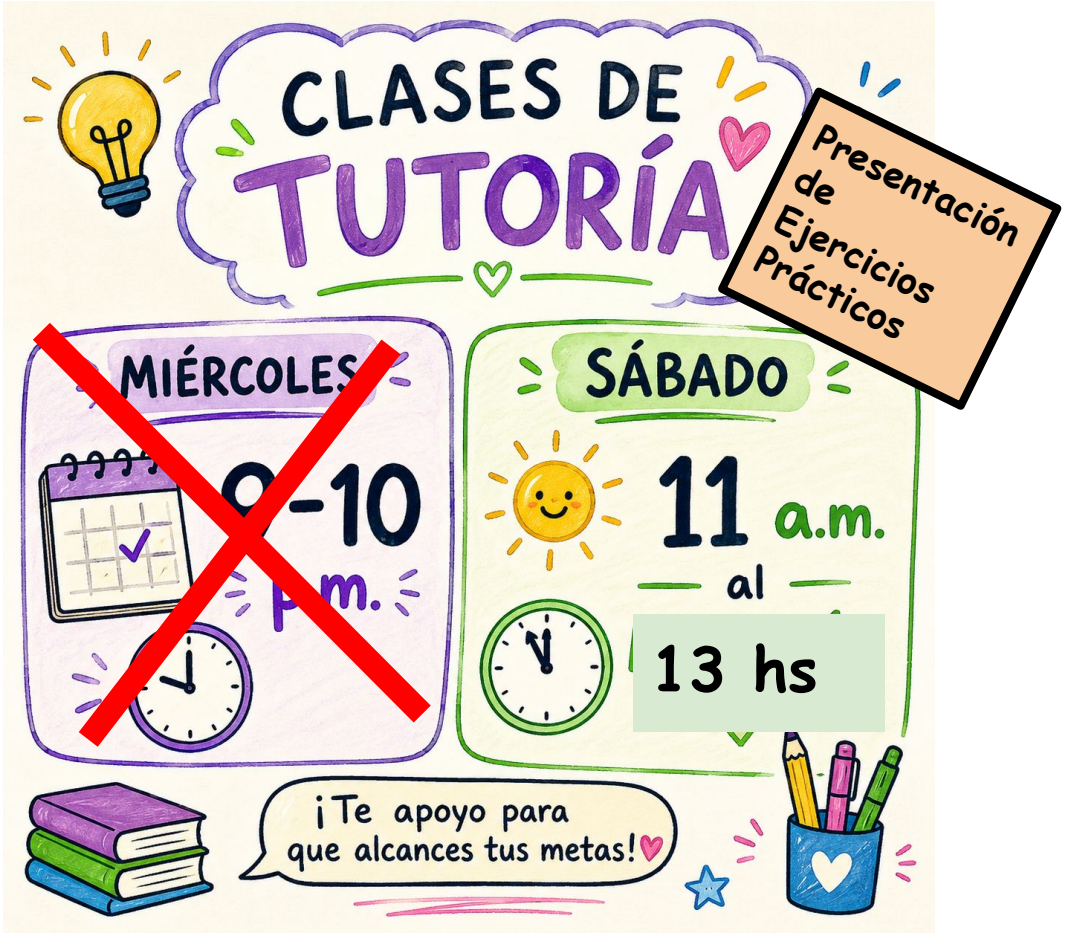
Tu meet es tu carta de presentación. ¡Cuidala!

3 Poder comunicar trabas, avances y dudas a tiempo, es preferible equivocarse aca que con una deadline.



Comunicar a tiempo = mejores soluciones, menos estrés.

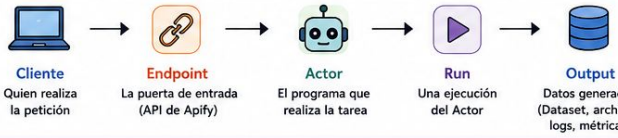
PENSAR. ORGANIZAR. COMUNICAR. = CRECER JUNTOS



[Inscripción a la reunión - Zoom]

CLASE ANTERIOR

VISION GENERAL



1. APIFY – LA PLATAFORMA

- Plataforma en la nube para ejecutar y gestionar automatizaciones, scraping y procesamiento de datos.
- Infraestructura escalable, segura y gestionada.
- Provee herramientas, UI, API, almacenamiento y monitoreo.

2. ACTOR – QUÉ ES

- Un Actor es un programa listo para realizar una tarea específica.
- Ejemplos: Web Scraper, PDF Text Extractor, Image Scraper, Google Search, etc.
- Cada Actor define el input que recibe y el output que produce.

3. RUN – UNA EJECUCIÓN

- Cuando ejecutás un Actor, Apify crea un Run.
- Cada Run tiene: input, estado (RUNNING / SUCCEEDED / FAILED), duración, consumo de recursos y output.
- Un mismo Actor puede tener muchos Runs.

4. ENDPOINT – LA PUERTA DE ENTRADA

- Es la URL (endpoint) que permite comunicarse con Apify mediante HTTP.
- Ejemplo de endpoint para ejecutar y obtener resultados al instante:

```
POST /v2/actors/{actorId}/run-sync-get-dataset-items
```

- Se envía el input (JSON) y se obtienen los resultados (dataset items).

5. CLIENTE – QUIEN REALIZA LA PETICIÓN

- Puede ser Make, Python, JavaScript/Node.js, cURL, Postman o tu propia aplicación.
- El cliente envía la petición al endpoint con el input necesario.
- Recibe el output y lo usa en su flujo o aplicación.

RESUMEN DE LA ARQUITECTURA

Cliente → Endpoint (API) → Actor → Run → Output (Datos)

Quien llama | Puerta de entrada | Programa / Herramienta | Ejecución | Resultados

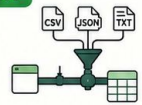
TIP CLAVE

- Actor = programa
- Run = ejecución
- Endpoint = URL/API
- Cliente = quien llama
- Output = resultados

CLASE DE HOY

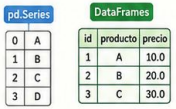
1. PANDAS — EJERCICIOS PROGRESIVOS

2.1 Estructuras de datos y pipeline



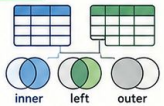
- Datos estructurados, semiestructurados y no estructurados
- Construcción del pipeline hacia `DataFrame`

2.2 Series y DataFrames



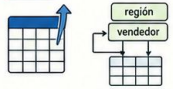
- Crear y explorar `Series` / `DataFrame`
- Selección: `loc` / `iloc`
- Índices y columnas
- Agregar, modificar y eliminar datos
- Ejercicio: productos + IVA + filtros

2.3 Combinar datos



- `pd.concat()`
- `pd.merge()`
- inner / left / outer
- Ejercicio: empleados + departamentos

2.4 Ordenamiento e índices



- `sort_values()`
- `sort_index()`
- `MultiIndex`
- Ejercicio: ventas + región + vendedor

2.5 Análisis de datos



- Estadísticas descriptivas
- `groupby()`
- *Pivoting*
- Muestreo

2. APIFY — RETOMAMOS EL CLIENTE



- Ejecutar un Actor
- Obtener resultados
- Procesar el output con Pandas

3. MAKE + REQUESTS



- Consumir APIs
- GET / POST
- JSON
- Requests vs. Make
- Llevar resultados a Pandas

4. DESAFÍO CREATIVO — UAV DRONE DATASET

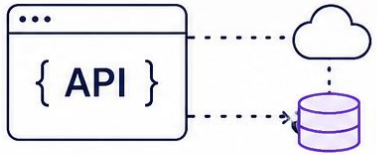


FLUJO DE LA CLASE

Pandas → Apify → Requests + Make → UAV Dataset

CRUD vs ENDPOINTS

¿SON LO MISMO?

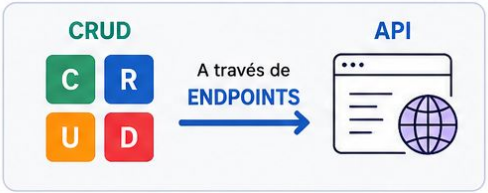


No exactamente. CRUD y Endpoints están relacionados, pero no son lo mismo.

CRUD = OPERACIONES QUE PUEDE HACER TU SISTEMA

CRUD significa las 4 operaciones básicas que puede realizar un sistema sobre los datos.

C	Create (Crear)	Crear un usuario
R	Read (Leer)	Obtener usuarios
U	Update (Actualizar)	Modificar un usuario
D	Delete (Eliminar)	Borrar un usuario



ENDPOINT = UNA URL DE ACCESO A UNA OPERACIÓN

Por ejemplo, una API REST podría tener estos endpoints:

MÉTODO HTTP	ENDPOINT	DESCRIPCIÓN	CORRESPONDE A
POST	`/usuarios`	Crea un nuevo usuario	C
GET	`/usuarios`	Obtiene todos los usuarios	R
GET	`/usuarios/15`	Obtiene el usuario con id 15	R
PUT	`/usuarios/15`	Actualiza el usuario con id 15	U
DELETE	`/usuarios/15`	Elimina el usuario con id 15	D

NO TODOS LOS ENDPOINTS SON CRUD

Existen endpoints que no representan directamente una operación CRUD.

- POST `/usuarios/15/reset-password`
- POST `/login`
- GET `/reportes/ventas`

Son endpoints, pero no forman parte de un CRUD.

EN RESUMEN:

CRUD = lógica / operaciones (qué puede hacer el sistema)

ENDPOINT = puerta de entrada (cómo accedemos a esas operaciones)

Los endpoints son la interfaz. El CRUD es la funcionalidad.

ESTRUCTURAS Y PIPELINE DE DATOS

Antes de analizar datos, primero entendemos qué tipo de datos tenemos

OBJETIVO

Convertir los datos en una estructura tabular homogénea que podamos analizar.

1. DATOS ESTRUCTURADOS

Tienen filas y columnas consistentes.
Ejemplos: CSV, Excel, bases de datos.

id	nombre	edad	ciudad
1	Ana	23	Salta
2	Luis	35	Córdoba
3	Eva	29	Buenos Aires

Fáciles de convertir directamente en un DataFrame.

2. DATOS NO ESTRUCTURADOS

No tienen esquema fijo.
Ejemplos: textos, imágenes, publicaciones, logs, audios, etc.

> "Excelente producto, llegó rápido y funciona muy bien."

LOG

Hay que extraer la información antes de analizarla.

3. DATOS SEMIESTRUCTURADOS

Tienen una estructura flexible.
Ejemplos: JSON, XML, HTML.

```
{
   "nombre": "Ana",
   "edad": 23,
   "ciudad": "Salta",
   "contacto": {
      "email": "ana@mail.com",
      "tel": "387-123456"
   }
}
```

JSON XML HTML

Se "aplanan" o normalizan antes de pasar a tabla.

EL PIPELINE

FUENTE → FORMATO → FORMA → TRATAMIENTO

1. FUENTE
Internet, archivos, bases de datos, APIs, etc.

2. FORMATO
CSV, JSON, HTML, XML, SQL, texto plano, etc.

3. FORMA
Lista, diccionario, o directamente DataFrame.

4. TRATAMIENTO
Limpieza, transformación, validación y preparación.

OBJETIVO FINAL
DataFrame listo para analizar

HERRAMIENTAS CLAVE

pd.read_csv()
Lee archivos CSV y los carga en un DataFrame.

pd.json_normalize()
Normaliza datos JSON (anidados) y los convierte en tabla.

pd.read_html()
Extrae tablas desde páginas HTML y las convierte en DataFrames.

EN RESUMEN

- ✓ Todo dato es estructurado, no estructurado o semiestructurado.
- ✓ El pipeline siempre busca llegar a una tabla homogénea.
- ✓ Estas funciones son tus puertas de entrada a los datos.

SERIES Y DATAFRAMES

Las dos estructuras principales de Pandas y cómo seleccionamos información.

1. SERIES (1D)

Estructura unidimensional con un índice (etiquetas).
Cada valor tiene una etiqueta asociada.

Índice	Valores
a	10
b	20
c	30
d	40

```
import pandas as pd

s = pd.Series(
    [10, 20, 30, 40],
    index=["a", "b", "c", "d"]
)
```

```
a    10
b    20
c    30
d    40
dtype: int64
```

← El índice puede ser texto, números, fechas, etc.

2. DATAFRAME (2D)

Estructura tabular bidimensional: filas y columnas.
Cada columna puede tener un tipo de dato distinto.

	nombre	edad	ciudad
0	Ana	23	Salta
1	Luis	35	Córdoba
2	Eva	29	CABA

↑ Índice (etiquetas de fila) ↑ Columnas (nombres)

```
df = pd.DataFrame({
  "nombre": ["Ana", "Luis", "Eva"],
  "edad": [23, 35, 29],
  "ciudad": ["Salta","Córdoba","CABA"]
})
```

IDEA CLAVE

- Series = 1 dimensión (como una columna)
- DataFrame = 2 dimensiones (filas y columnas)
- El índice permite identificar y alinear datos automáticamente.

3. INDEXING Y SELECCIÓN EN DATAFRAMES

Pandas ofrece varias formas de acceder a los datos. Las más importantes son loc, iloc y los filtros booleanos.

3.1 loc[] → selección por ETIQUETA (índice)

Permite seleccionar filas y columnas usando las etiquetas del índice.
Incluye el límite final (comportamiento "cerrado").

Índice	nombre	edad	ciudad
f1	Ana	23	Salta
f2	Luis	35	Córdoba
f3	Eva	29	CABA

```
df.loc["f1"]
# fila con etiqueta f1
```

```
nombre    Ana
edad       23
ciudad  Salta
Name: f1, dtype: object
```

```
df.loc["f1":"f2", "edad"]
# rango de etiquetas (incluye f2)
```

```
f1    23
f2    35
Name: edad, dtype: int64
```

```
df.loc[["f1","f3"],
 ["nombre","ciudad"]]
# varias filas y columnas
```

	nombre	ciudad
f1	Ana	Salta
f3	Eva	CABA

3.2 iloc[] → selección por POSICIÓN

Usa la posición numérica (como en las listas de Python).
El límite final **NO** se incluye (comportamiento "abierto").

```
df.iloc[0]
# primera fila (posición 0)
```

```
nombre    Ana
edad       23
ciudad  Salta
Name: f1, dtype: object
```

```
df.iloc[0:2, 1]
# filas 0 y 1, columna 1 (edad)
```

```
0    23
1    35
Name: edad, dtype: int64
```

```
df.iloc[:, 0:2]
# todas las filas, columnas 0 y 1
```

	nombre	edad
f1	Ana	23
f2	Luis	35
f3	Eva	29

3.3 Selección de columnas

`df["col"]` o `df.col`
Devuelve una Series (columna única)

```
df["nombre"]
```

```
f1    Ana
f2    Luis
f3    Eva
Name: nombre, dtype: object
```

`df[["col"]]`
Devuelve un DataFrame (columna como tabla)

```
df[["nombre"]]
```

	nombre
f1	Ana
f2	Luis
f3	Eva

3.4 Boolean indexing (filtros)

Filtra filas que cumplan una condición.

```
df[df["edad"] > 25]
```

	nombre	edad	ciudad
f2	Luis	35	Córdoba
f3	Eva	29	CABA

```
df[(df["edad"] >= 25) & (df["ciudad"] == "CABA")]
```

	nombre	edad	ciudad
f3	Eva	29	CABA

3.5 Acceso a un solo valor (rápido)

`at[]` → por etiqueta
Muy eficiente para un único valor.

```
df.at["f2", "edad"]
```

```
35
```

`iat[]` → por posición
Muy eficiente para un único valor.

```
df.iat[1, 1]
```

```
35
```

Usa `at`/`iat` solo cuando necesites un único valor.
Para subconjuntos usa `loc`/`iloc`.

3.6 Rangos y slices: diferencia clave entre loc y iloc

loc incluye el límite final

f1 f2 f3 f4 f5
0 1 2 3 4

```
df.loc["f1":"f3"]
```

	nombre	edad	ciudad
f1	Ana	23	Salta
f2	Luis	35	Córdoba
f3	Eva	29	CABA

iloc NO incluye el límite final

f1 f2 f3 f4 f5
0 1 2 3 4

```
df.iloc[0:2]
```

	nombre	edad	ciudad
f1	Ana	23	Salta
f2	Luis	35	Córdoba

RESUMEN RÁPIDO

- **loc** → por etiquetas (incluye el final)
- **iloc** → por posición (excluye el final)
- **[]** → seleccionar columnas
- **Boolean indexing** → filtrar con condiciones
- **at / iat** → acceso rápido a un solo valor

UNIR Y ORGANIZAR DATAFRAMES

Cuando trabajamos con datos reales, la información suele estar distribuida en varias tablas.
Estas herramientas nos permiten combinar, alinear y ordenar los datos para analizarlos mejor.

① CONCAT(): APILAR TABLAS

Sirve para unir DataFrames con la misma estructura (mismas columnas). Se usa para apilar filas o columnas.

Ejemplo: apilar por filas (axis=0)

ENERO

id	ventas
1	120
2	80

FEBRERO

id	ventas
1	150
2	90

MARZO

id	ventas
1	130
2	110

Código:

```
df_final = pd.concat(
  [df_enero, df_febrero, df_marzo],
  ignore_index=True
)
```

RESULTADO (concat axis=0)

id	ventas	mes
1	120	enero
2	80	enero
1	150	febrero
2	90	febrero
1	130	marzo
2	110	marzo

Se agrega la columna "mes" para identificar el origen.

Ejemplo: apilar por columnas (axis=1)

NOTAS 1º PARCIAL

id	mate	lengua
1	8	7
2	6	9

NOTAS 2º PARCIAL

id	mate	lengua
1	9	8
2	7	6

ASISTENCIA (%)

id	asistencia
1	90
2	80

RESULTADO (concat axis=1)

id	mate	lengua	mate	lengua	asistencia
1	8	7	9	8	90
2	6	9	7	6	80

Código:

```
df_total = pd.concat(
  [df_parcial1, df_parcial2, df_asistencia], axis=1
)
```

TIP
Concat es ideal cuando las tablas tienen la MISMA estructura.

② MERGE(): UNIR TABLAS POR UNA CLAVE

Permite combinar información usando una columna en común, similar a un JOIN en SQL.

TABLA CLIENTES

id	nombre	ciudad
1	Ana	Salta
2	Luis	Córdoba
3	Eva	Mendoza
4	Juan	Rosario

TABLA PEDIDOS

cliente_id	fecha	monto
1	01/05	100
1	10/05	50
2	05/05	200
4	12/05	80

MERGE por id (left_on="id", right_on="cliente_id")
how="left"

id	nombre	ciudad	cliente_id	fecha	monto
1	Ana	Salta	1	01/05	100
1	Ana	Salta	1	10/05	50
2	Luis	Córdoba	2	05/05	200
3	Eva	Mendoza	NaN	NaN	NaN
4	Juan	Rosario	4	12/05	80

Código:

```
pd.merge(clientes, pedidos,
    left_on="id", right_on="cliente_id", how="left")
```

TIPOS DE MERGE (how)

- inner → solo las filas que coinciden.
- left → todas las filas de la izquierda.
- right → todas las filas de la derecha.
- outer → todas las filas de ambas tablas.

EJEMPLO SQL EQUIVALENTE (para el merge anterior)

```
SELECT  c.id, c.nombre, c.ciudad, p.fecha, p.monto
FROM    clientes c
LEFT JOIN  pedidos p
    ON c.id = p.cliente_id;
```

③ JOIN(): UNIR POR ÍNDICE

Útil cuando las tablas comparten el mismo índice. Es una forma simplificada de merge.

DF A (índice: id)

id	nombre
1	Ana
2	Luis
3	Eva

join

DF B (índice: id)

id	total_compras
1	150
2	200
3	0

RESULTADO (join)

id	nombre	total_compras
1	Ana	150
2	Luis	200
3	Eva	0

Código:

```
df_a.join(df_b,
  how="left")
```

El índice funciona como la "clave" de unión.

④ ORGANIZAR: ORDENAR E ÍNDICES AVANZADOS

ORDENAR POR VALORES

Ejemplo DataFrame original

id	nombre	edad
3	Eva	29
1	Ana	23
2	Luis	35

Código:

```
df.sort_values("edad",
    assending=False)
```

Resultado

id	nombre	edad
2	Luis	35
3	Eva	29
1	Ana	23

ORDENAR POR ÍNDICE (alfabético o numérico)

Código:

```
df.sort_index()
```

Resultado

id	nombre	edad
1	Ana	23
2	Luis	35
3	Eva	29

MULTIÍNDEX: ÍNDICES JERÁRQUICOS

Ejemplo: Ventas por región y ciudad

región	ciudad	producto	ventas
Norte	Salta	Teclado	120
	Salta	Mouse	80
	Jujuy	Teclado	90
Sur	Neuquén	Teclado	110
	Bariloche	Mouse	150

Código:

```
df.set_index(["región", "ciudad"],
  inplace=True)
```

Ahora el índice tiene dos niveles:

ÍNDICE (2 niveles)

región	ciudad	producto	ventas
Norte	Salta	Teclado	120
Norte	Salta	Mouse	80
Norte	Jujuy	Teclado	90
Sur	Neuquén	Teclado	110
Sur	Bariloche	Mouse	150

nivel 0 (región) →
nivel 1 (ciudad) →

IDEA CLAVE: Unir y organizar correctamente los datos es fundamental para analizarlos sin errores.
Concat para apilar, merge para relacionar, join para usar el índice y ordenar para entender mejor.

ETL CON PANDAS: DE DATOS CRUDOS A DATOS LISTOS

ETL = EXTRACT → TRANSFORM → LOAD

Objetivo: convertir datos crudos en información limpia, consistente y lista para analizar.

1) EXTRACT

Extraer los datos

FUENTES

- CSV
- Excel
- JSON
- Base de datos SQL
- API

Ejemplo con Pandas:

```
import pandas as pd

# CSV
df = pd.read_csv("ventas.csv")

# Excel
df = pd.read_excel("ventas.xlsx")

# JSON
df = pd.read_json("ventas.json")
```

Inspección inicial

- `df.head()` → primeras filas
- `df.shape` → (filas, columnas)
- `df.info()` → tipos y nulos

DATOS CRUDOS (ventas.csv)

id	cliente	edad	ingreso	fecha	ciudad
1	Ana	23	"1000"	01/02/2024	Salta
2	Luis	NaN	"2000"	03/02/2024	Córdoba
2	Luis	NaN	"2000"	03/02/2024	Córdoba
3	Eva	29	"1500"	05/02/2024	NaN
4	Matías	31	"1200"	07/02/2024	Mendoza

Problemas:

- ✗ ingreso es texto
- ✗ valores faltantes
- ✗ fila duplicada
- ✗ fecha como texto

2) TRANSFORM

Transformar / limpiar / crear información

1 Corregir tipos de datos

123 numérico

fecha

```
df["ingreso"] = pd.to_numeric(df["ingreso"], errors="coerce")
df["fecha"] = pd.to_datetime(df["fecha"], errors="coerce")
```

Antes

ingreso	fecha
"1000"	01/02/2024
"2000"	03/02/2024

Después

ingreso	fecha
1000.0	2024-02-01
2000.0	2024-02-03

2 Tratar valores faltantes

isna().sum()

```
df["edad"] = df["edad"].fillna(df["edad"].median())
df["ciudad"] = df["ciudad"].fillna(df["ciudad"].mode()[0])
```

Antes

edad	ciudad
NaN	Córdoba
29	NaN

Después

edad	ciudad
27.0	Córdoba
29.0	Córdoba

3 Eliminar duplicados

duplicated()

```
df = df.drop_duplicates()
```

Antes 5 filas → Después 4 filas

4 Crear nuevas variables (Feature Engineering)

```
df["ingreso_anual"] = df["ingreso"] * 12
df["mes"] = df["fecha"].dt.month
df["grupo_edad"] = pd.cut(
    df["edad"],
    bins=[0,18,35,60,100],
    labels=["menor", "joven", "adulto", "mayor"]
)
```

Resultado parcial

edad	ingreso	mes	ingreso_anual	grupo_edad
23.0	1000.0	2	12000.0	joven
27.0	2000.0	2	24000.0	joven
29.0	1500.0	2	18000.0	joven
31.0	1200.0	2	14400.0	joven

5 Seleccionar columnas relevantes

```
df = df[["id", "cliente", "edad", "ciudad", "ingreso",
         "ingreso_anual", "mes", "grupo_edad"]]
```

menos = mejor

Quitamos columnas innecesarias para quedarnos con lo que necesitamos.

3) LOAD

Cargar los datos transformados

Destino

- Base de datos / Data Warehouse
- Archivo limpio (CSV / Parquet)
- Data Lake
- DataFrame para análisis

Ejemplo con Pandas:

```
df.to_csv("ventas_limpias.csv", index=False)

# También podría ser:
# df.to_parquet("ventas_limpias.parquet")
```

DATOS LISTOS PARA ANALIZAR

id	cliente	edad	ciudad	ingreso	ingreso_anual	mes	grupo_edad
1	Ana	23.0	Salta	1000.0	12000.0	2	joven
2	Luis	27.0	Córdoba	2000.0	24000.0	2	joven
3	Eva	29.0	Córdoba	1500.0	18000.0	2	joven
4	Matías	31.0	Mendoza	1200.0	14400.0	2	joven

Datos limpios, consistentes y listos para análisis, dashboards o modelos.

EXTRACT Obtenemos los datos desde la fuente.

TRANSFORM Los limpiamos, corregimos y generamos información útil.

LOAD Guardamos los datos en el destino elegido.

ANÁLISIS Generamos valor: métricas, gráficos, modelos.

Regla de oro: ¡Siempre trabajar sobre una copia de los datos!

FALTANTES, DUPLICADOS Y FEATURE ENGINEERING

1. VALORES FALTANTES

- Detectamos los faltantes

```
df.isna().sum()
```

Columna	Faltantes
nombre	0
edad	12
ingreso	5
ciudad	3

Edad tiene 12 valores faltantes

¿QUÉ HACEMOS?

Eliminar filas con faltantes

```
df.dropna()
```

Completar con la mediana (numérico)

```
df["edad"] = df["edad"].fillna(
    df["edad"].median()
)
```

Completar con la moda (categórico)

```
df["ciudad"] = df["ciudad"].fillna(
    df["ciudad"].mode()[0]
)
```

IDEA CLAVE

Elegimos la estrategia según el tipo de dato y el contexto del problema.

2. DUPLICADOS

- Detectar duplicados

```
df.duplicated().sum()
```

Ejemplo: 8 duplicados

- Eliminar duplicados

```
df.drop_duplicates()
```

- Definir clave de comparación

```
df.drop_duplicates(subset=["dni"])
# mantiene el primero por defecto
```

EJEMPLO

ANTES

dni	nombre	edad
123	Ana	23
456	Luis	35
123	Ana	23
789	Eva	29
456	Luis	35

DESPUÉS

dni	nombre	edad
123	Ana	23
456	Luis	35
789	Eva	29

BUENAS PRÁCTICAS

- Revisar siempre antes de eliminar.
- Guardar una copia antes de transformar.
- Entender el negocio detrás de los datos.

3. FEATURE ENGINEERING

- Crear nuevas variables útiles

```
df["grupo_edad"] = pd.cut(
    df["edad"],
    bins=[0,18,35,60,100],
    labels=["menor","joven","adulto","mayor"]
)
```

Ejemplo: agrupar edades

Resultado

edad	grupo_edad
15	menor
25	joven
42	adulto
72	mayor

- Codificar variables categóricas

```
pd.get_dummies(df, columns=["ciudad"], drop_first=True)
```

Antes

ciudad
Salta
Córdoba
Buenos Aires
Salta

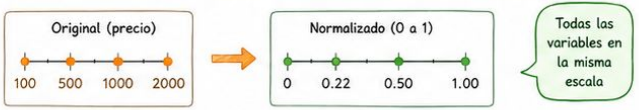
Despvés (one-hot encoding)

ciudad_Córdoba	ciudad_Salta
0	1
1	0
0	0
0	1

Se convierte la categoría en columnas numéricas

- Escalar variables numéricas

```
df["precio_norm"] = (df["precio"] - df["precio"].min()) / \
                    (df["precio"].max() - df["precio"].min())
```



Todas las variables en la misma escala

DATOS CRUDOS → **LIMPIAR**
- Faltantes
- Duplicados
- Tipos de dato

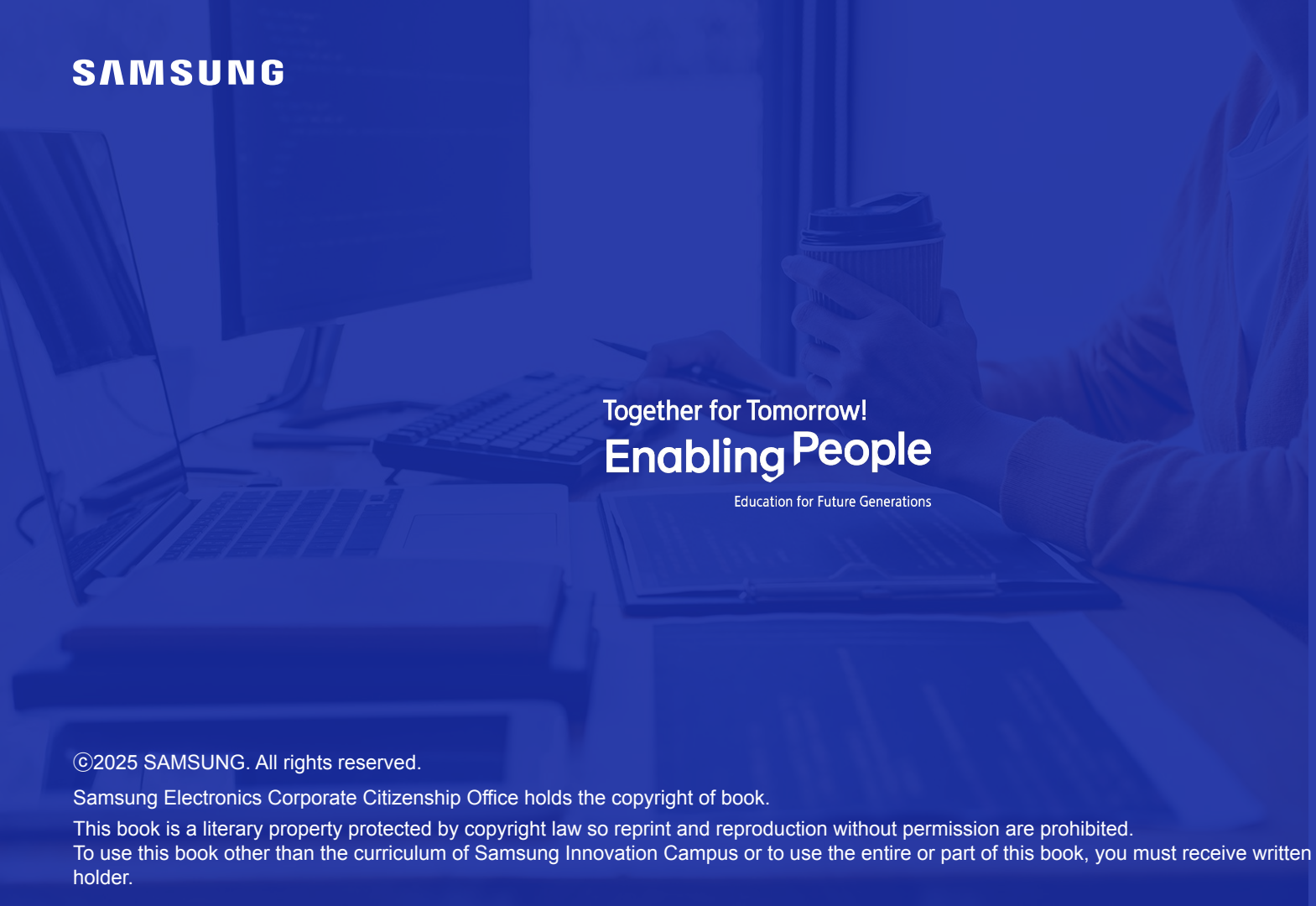
→ **CORREGIR**
- Formatos
- Errores
- Inconsistencias

→ **CREAR VARIABLES**
- Nuevas
- Relevantes
- Informativas

→ **CODIFICAR**
- One-hot
- Label encoding
- Otras técnicas

→ **ESCALAR**
- Normalización
- Estandarización
- Otras técnicas

→ **DATOS LISTOS PARA ANALIZAR**



ⓒ2025 SAMSUNG. All rights reserved.

Samsung Electronics Corporate Citizenship Office holds the copyright of book.

This book is a literary property protected by copyright law so reprint and reproduction without permission are prohibited.
To use this book other than the curriculum of Samsung Innovation Campus or to use the entire or part of this book, you must receive written consent from copyright holder.